

ASBDEMO PROJECT DOCUMENTATION

Terraform-provisioned Azure infrastructure, Spring Boot microservices, Azure Service Bus messaging, AKS GitOps delivery, Gateway API ingress, and observability. Repository reviewed: azureservicebus

Document generated: 2026-05-02

Environment described: azure-dev, application namespace asbdemo, Azure region southindia

Primary source paths: infra/, app/, gitops/, .github/workflows/, azure-pipelines/

TABLE OF CONTENTS

- Executive Summary
- Architecture Overview
- Repository Structure
- Azure Infrastructure Provisioning with Terraform
- Spring Boot Microservices
- Azure Service Bus Messaging Flow
- Containerization
- AKS Workload Deployment with GitOps
- Gateway API and Traffic Management
- Progressive Delivery with Argo Rollouts
- Monitoring, Logging, and Tracing
- CI/CD Automation
- Security and Identity Model
- Operational Runbook
- Current Implementation Notes and Recommended Next Steps

1. EXECUTIVE SUMMARY

This project is an end-to-end cloud-native demonstration that connects two Java 21 Spring Boot microservices through Azure Service Bus and runs them on Azure Kubernetes Service (AKS). The infrastructure is declared through Terraform, delivery is handled through GitOps using ArgoCD, progressive deployment behavior is managed by Argo Rollouts, and runtime visibility is provided through Prometheus, Grafana, Jaeger, Elasticsearch, Kibana, and Fluentd.

The application domain is intentionally small so the platform patterns are easy to see. `orders-service` accepts an order creation request over HTTP and publishes an `OrderEvent` to an Azure Service Bus queue. `payments-service` consumes the same queue and records message processing success or failure. Both services expose Spring Actuator health and Prometheus endpoints, emit structured logs outside local mode, and use Azure workload identity rather than connection strings in AKS.

The project demonstrates several production-aligned practices: remote Terraform execution through HCP Terraform, Azure managed identities with federated Kubernetes service accounts, ACR image publishing, app-of-apps GitOps, canary and blue-green rollout strategies, Gateway API ingress, Prometheus alerting rules, and JSON log shipping into Elasticsearch.

2. ARCHITECTURE OVERVIEW

Developer / CI

|

| app or infra change

v

GitHub Actions or Azure Pipelines

|-- Terraform workflow: fmt, validate, plan, apply through HCP Terraform

|-- App workflow: build images, Trivy scan, push to ACR, bump GitOps image tags

v

Git repository as source of truth

|

| ArgoCD watches gitops/

v

AKS cluster

|-- Platform apps: Gateway API CRDs, NGINX Gateway Fabric, cert-manager,

| Argo Rollouts, kube-prometheus-stack, Jaeger, ECK, Elasticsearch, Kibana, Fluentd

|

|-- Workloads in namespace asbdemo

|-- orders-service: HTTP API, Service Bus sender, canary rollout

|-- payments-service: Service Bus processor, blue-green rollout

|

v

Azure Service Bus namespace and orders queue

The infrastructure and runtime system are separated into clear concerns. Terraform creates Azure resources and Kubernetes-ready identities. GitOps deploys in-cluster platform add-ons and application workloads. CI/CD connects application source changes to the GitOps layer by publishing immutable image tags and updating the workload customization.

KEY TECHNOLOGY CHOICES

Area | Implementation | Purpose

Infrastructure as Code | Terraform >= 1.9.0, HCP Terraform workspace [vinod-techops-org/azure-dev](https://github.com/vinod-techops-org/azure-dev) | Reproducible provisioning and remote state/execution.

Cloud platform | Azure, regionsouthindia | Hosts AKS, ACR, Service Bus, Key Vault, and Log Analytics.

Kubernetes | AKS with Azure CNI and Cilium network policy/data plane | Container orchestration for microservices and platform add-ons.

Messaging | Azure Service Bus Basic SKU queue namedorders | Point-to-point asynchronous communication between services.

Application framework | Spring Boot 3.3.5, Java 21, Maven multi-module project | REST API, Service Bus clients, health checks, metrics.

Identity | AKS OIDC issuer plus Azure Workload Identity | Passwordless Azure authentication from Kubernetes pods.

GitOps | ArgoCD app-of-apps with Kustomize overlays | Cluster state reconciled from Git.

Progressive delivery | Argo Rollouts | Canary for orders and blue-green for payments.

Ingress | Gateway API with NGINX Gateway Fabric | HTTP routing to the orders API.

Observability | Prometheus, Grafana, Jaeger, ECK Elasticsearch/Kibana, Fluentd | Metrics,

dashboards, alerts, traces, and logs.

3. REPOSITORY STRUCTURE

Path | Description

app/ | Maven parent project containing orders-service and payments-service, Dockerfiles, local Docker Compose, and application configuration.

infra/envs/azure-dev/ | The active Terraform environment stack. It wires together all Azure modules for the dev environment and uses the HCP Terraform cloud block.

infra/modules/ | Reusable Terraform modules for network, AKS, ACR, Key Vault, monitoring, Service Bus, and workload identity.

gitops/bootstrap/argocd/ | ArgoCD Helm values, bootstrap script, and root ArgoCD Applications.

gitops/platform/apps/ | ArgoCD Applications for platform add-ons such as Gateway API CRDs, NGINX Gateway Fabric, cert-manager, Argo Rollouts, Prometheus, Jaeger, and ECK.

gitops/platform/elastic/ | Elasticsearch, Kibana, Fluentd configuration, RBAC, and DaemonSet resources.

gitops/workloads/ | Application manifests for orders, payments, environment overlay, Gateway, PrometheusRule, and Grafana dashboard ConfigMap.

.github/workflows/ | GitHub Actions workflows for Terraform and application build/scan/push/GitOps bump.

azure-pipelines/ | Azure DevOps mirror pipelines for Terraform and application delivery.

docs/ | Operational runbook and generated project documentation.

4. AZURE INFRASTRUCTURE PROVISIONING WITH TERRAFORM

The Terraform environment under `infra/envs/azure-dev` is the central composition layer. It defines the resource group and invokes reusable modules for the core Azure landing zone and workload dependencies. Naming is controlled through locals using the format `<resource-type>-asbdemo-dev-sin`, with a no-dash variant for globally constrained resources such as ACR.

4.1 TERRAFORM PROVIDERS AND REMOTE EXECUTION

The stack requires Terraform $\geq 1.9.0$ and uses the `azurerm`, `azuread`, and `random` providers. The Terraform cloud block points to `organizationvinod-techops-org` and `workspaceazure-dev`, so local commands stream remote HCP Terraform runs rather than executing plans locally.

Provider features intentionally set `prevent_deletion_if_contains_resources = false` for resource groups and configure Key Vault soft-delete recovery/purge behavior. The runbook explains one-time OIDC federation setup for HCP Terraform so the workspace can authenticate to Azure without storing a client secret.

4.2 RESOURCE GROUP AND NAMING

The environment provisions one main resource group named `drg-asbdemo-dev-sin`. Tags are consistently applied with keys for application, environment, Terraform ownership, repository, and HCP workspace. The Service Bus namespace includes a random four-character suffix to satisfy Azure's global uniqueness requirement.

4.3 NETWORKING MODULE

The network module creates a virtual network with address space 10.40.0.0/16 and three subnets:

- aks-system:10.40.1.0/24, used by the AKS system node pool.
- aks-user:10.40.2.0/24, configured with AKS pod subnet delegation to Microsoft.ContainerService/managedClusters.
- pe:10.40.10.0/24, reserved for private endpoint style expansion.

A network security group is associated with the AKS system and user subnets. The pod delegation is important because AKS attaches service association links to pod subnets; declaring the delegation prevents Terraform drift during refresh and update operations.

4.4 MONITORING MODULE

The monitoring module creates an Azure Log Analytics Workspace using SKUPerGB2018 and default retention of 30 days. The AKS module connects Container Insights to this workspace through the oms_agent block with managed identity authentication enabled.

4.5 KEY VAULT MODULE

The Key Vault module provisions a standard SKU vault with Azure RBAC authorization enabled. Soft delete retention defaults to seven days, purge protection is disabled for dev convenience, and public network access is currently enabled. The vault is intended as the central secure store for pipeline tokens and future cluster secrets.

4.6 AZURE CONTAINER REGISTRY MODULE

A Basic SKU Azure Container Registry is created with admin user disabled. The AKS kubelet identity is granted AcrPull on this registry by the AKS module, allowing Kubernetes nodes to pull images without static registry credentials.

4.7 AKS MODULE

The AKS cluster is configured with workload identity and OIDC issuer enabled. It uses a system-assigned control plane identity, Azure CNI networking, Cilium network policy, Cilium data plane, Standard load balancer, and outbound type load balancer. The service CIDR is 10.200.0.0/16 with DNS service IP 10.200.0.10.

Node pools are split by function:

Pool	Mode	Default size	Role
system	System	Standard_B2als_v2, one node	Runs critical cluster add-ons only.
user	User	Standard_D2s_v4, autoscale min 3 max 5	Runs application and platform workloads, labeled workload=apps.

The cluster uses automatic patch upgrades and ignores drift on Kubernetes version and node

counts, which is useful when AKS or autoscaling changes these values outside Terraform.

4.8 AZURE SERVICE BUS MODULE

The Service Bus module creates a Basic SKU namespace with minimum TLS version 1.2. Public network access and local authentication are currently enabled, but the application manifests are designed to authenticate with Azure AD through workload identity. The environment creates a single queue named orders.

Queue behavior:

- max_delivery_count = 10, after which messages move toward dead-letter handling.
- dead_lettering_on_message_expiration = true.
- default_message_ttl = P14D, so messages expire after 14 days.
- lock_duration = PT1M, giving consumers a one-minute lock window.

4.9 WORKLOAD IDENTITY MODULE

Two user-assigned managed identities are created. Each identity has a federated identity credential whose subject matches a Kubernetes service account:

Service	Kubernetes ServiceAccount	Azure Role	Scope
orders-service	asbdemo/orders-sa	Azure Service Bus Data Sender	ordersqueue resource ID
payments-service	asbdemo/payments-sa	Azure Service Bus Data Receiver	ordersqueue resource ID

This is a least-privilege split: the producer can send but not receive, and the consumer can receive but not send.

5. SPRING BOOT MICROSERVICES

The app/directory is a Maven multi-module project. The parent POM defines Java 21, Spring Boot 3.3.5, Spring Cloud Azure dependency management 5.18.0, and the Logstash Logback encoder version. Both child services include Spring Web, Actuator, Azure Service Bus client, Azure Identity, Micrometer Prometheus registry, structured logging, and Spring Boot test dependencies.

5.1 ORDERS-SERVICE

orders-service is the producer. It exposes one primary endpoint:

POST /orders

Content-Type: application/json

```
{
  "amount": 125.50,
  "customer": "customer-name"
}
```

The controller creates an `OrderEvent` containing a generated UUID order ID, amount, customer, and creation timestamp. It then calls `OrderPublisher.publish` and returns HTTP 202 Accepted with the generated order ID.

The `OrderPublisher` serializes the event with Jackson, creates a `ServiceBusMessage`, sets the content type to `application/json`, sets the message ID to the order ID, sends the message, and logs the publish event. The service also registers a custom Micrometer counter named `orders_published_total`.

Service Bus connectivity is defined in `ServiceBusConfig`. It builds a `ServiceBusSenderClient` using `DefaultAzureCredentialBuilder`, the configured namespace FQDN, and the queue name. In AKS, `DefaultAzureCredential` resolves through workload identity. Locally, it can resolve through service principal environment variables provided in Docker Compose.

5.2 PAYMENTS-SERVICE

`payments-service` is the consumer. It creates a `ServiceBusProcessorClient` for the same queue using `DefaultAzureCredentialBuilder`. The processor is started from an `ApplicationRunner` when the Spring Boot application starts.

The processor uses `maxConcurrentCalls(4)`, allowing up to four messages to be processed concurrently. Each message body is deserialized into the `sharedOrderEvent` shape. Successful messages are logged, counted through `orders_processed_total`, and completed with `ctx.complete()`. Failed messages are logged, counted through `orders_failed_total`, and abandoned with `ctx.abandon()`, allowing Service Bus redelivery until max delivery behavior is reached.

5.3 COMMON RUNTIME CONFIGURATION

Both services run on port 8080 and enable graceful shutdown. Actuator exposes `health`, `info`, `prometheus`, and `metrics`. Kubernetes probes use the actuator liveness and readiness endpoints. Metrics include an `application` tag set to the Spring application name.

Service Bus configuration is supplied through environment variables:

- `SERVICEBUS_NAMESPACE_FQDN`, for example `sb-asbdemo-dev-sin-znsc.servicebus.windows.net`.
- `SERVICEBUS_QUEUE_NAME`, defaulting to `orders`.

Logging switches by profile. In the `local` profile, logs are console-pattern text. In non-local profiles, logs are JSON using `LogstashEncoder`, with custom `app` field and MDC keys for `traceId` and `spanId`.

6. AZURE SERVICE BUS MESSAGING FLOW

- An external caller sends `POST /orders` through the Gateway API route or directly to the service inside the cluster.
- `orders-service` validates and accepts the request body, creates an `OrderEvent`, and sends it to the `orders` queue.
- Azure Service Bus stores the message durably in the queue until a consumer locks and completes

it.

- payments-service has an active processor client listening on the queue.
- The processor receives a message, deserializes it, logs the order, increments `orders_processed_total`, and completes the message.
- If processing fails, the consumer increments `orders_failed_total` and abandons the message, allowing retry according to Service Bus delivery behavior.

The queue is used as a decoupling layer. orders-service only needs to publish successfully; it does not depend on payments-service being healthy at the same moment. This is the central resilience pattern demonstrated by the project.

7. CONTAINERIZATION

Each service has a Dockerfile using a two-stage build. The build stage uses `maven:3.9.9-eclipse-temurin-21`, copies the parent and child POM files first, resolves dependencies with BuildKit cache mounts, then copies source and packages the selected Maven module. The runtime stage uses `eclipse-temurin:21-jre`.

Both runtime images install `curl` and CA certificates, download the OpenTelemetry Java agent version 2.11.0, create a non-root user with UID 10001, expose port 8080, and run with:

```
java -javaagent:/opt/opentelemetry-javaagent.jar \  
-XX:+UseContainerSupport \  
-XX:MaxRAMPercentage=75 \  
-jar /app/app.jar
```

Local Docker Compose builds both images from the `app/context`. It maps orders to host port 8080 and payments to host port 8081. It expects Azure credentials and Service Bus settings in a local `envfile` based on `env.example`. The orders container waits for the payments health check before starting, which is convenient for a local demo even though the production queue decouples them.

8. AKS WORKLOAD DEPLOYMENT WITH GITOPS

The workload GitOps entrypoint is `gitops/workloads/envs/dev/kustomization.yaml`. It sets namespace `asbdemo`, includes the namespace, Gateway, PrometheusRule, Grafana dashboard, orders service resources, and payments service resources. It also controls the currently deployed image tags through Kustomize image overrides.

At the time this document was generated, the dev overlay points both images at `tag1d986ecinacrasbdemovsin.azurecr.io`. CI updates those tags after successful image builds on pushes to main.

8.1 ORDERS-SERVICE KUBERNETES RESOURCES

- ServiceAccount `orders-sa` annotated with the Azure managed identity client ID for the sender identity.
- Rollout `orders-service` with two replicas and a canary strategy.

- Service orders-serviceexposing port 80 to target port 8080.
- ServiceMonitor orders-servicescrapping/actuator/prometheusevery 15 seconds.
- AnalysisTemplate success-ratequerying Prometheus for HTTP success rate during canary rollout.

8.2 PAYMENTS-SERVICE KUBERNETES RESOURCES

- ServiceAccount payments-saannotated with the Azure managed identity client ID for the receiver identity.
- Rollout payments-servicewith one replica and a blue-green strategy.
- Service payments-serviceas the active service.
- Service payments-service-previewas the preview service for blue-green promotion.
- ServiceMonitor payments-servicescrapping/actuator/prometheusevery 15 seconds.

8.3 POD RUNTIME SETTINGS

Both services request 100mCPU and 384Mi memory, with limits of 500mCPU and 512Mi memory. They define startup, readiness, and liveness probes against Spring Boot actuator endpoints. Each pod has a 40-second termination grace period and a five-second pre-stop sleep to give the service time to drain during rollout transitions.

Both deployments set OpenTelemetry environment variables that send traces over OTLP gRPC to jaeger.jaeger.svc.cluster.local:4317. Metrics and logs exporters are disabled in the OpenTelemetry agent configuration because Prometheus scrapes metrics directly and Fluentd ships container logs.

9. GATEWAY API AND TRAFFIC MANAGEMENT

The platform installs Gateway API standard CRDs and NGINX Gateway Fabric. The workload overlay defines an NginxProxy, a Gateway, and an HTTPRoute. The Gateway listens on HTTP port 80 and allows routes from the same namespace. The HTTPRoute is named orders, uses host name orders.asbdemo.local, matches path prefix /orders, and sends traffic to orders-service on port 80.

NGINX Gateway Fabric is currently configured with ClusterIP services. That fits an internal or port-forwarded dev setup. For internet-facing access, the NGINX service or Gateway infrastructure would need to expose a public load balancer or be integrated with an ingress/load-balancer pattern appropriate for the target environment.

10. PROGRESSIVE DELIVERY WITH ARGO ROLLOUTS

10.1 ORDERS-SERVICE CANARY

orders-service uses a canary rollout with weight steps of 25%, 50%, and 75%, each followed by a 30-second pause. Starting at step 1, Argo Rollouts runs the success-rateAnalysisTemplate. The Prometheus query calculates the ratio of non-5xx HTTP requests to total HTTP requests for the service over two minutes and treats empty data as success with vector(1). The rollout succeeds when the result is at least 0.95 and fails after three failed checks.

10.2 PAYMENTS-SERVICE BLUE-GREEN

payments-service uses blue-green deployment with an active service and preview service. Auto-promotion is disabled, so a new version can be observed through the preview service before being promoted. The old replica set is retained for 60 seconds after promotion before scale-down.

11. MONITORING, LOGGING, AND TRACING

11.1 PROMETHEUS AND GRAFANA

The platform app installs `skube-prometheus-stack` into namespace `monitoring`. The Helm values are tuned for a dev cluster: default rules are disabled, Prometheus retention is six hours, persistent storage is disabled, admission webhooks and TLS for the operator are disabled, and Grafana uses a ClusterIP service with dashboard sidecar enabled across all namespaces.

The application ServiceMonitors make Prometheus scrape both Spring Boot apps at `/actuator/prometheus`. Custom application metrics include:

- `orders_published_total`, incremented by orders-service after a successful publish call.
- `orders_processed_total`, incremented by payments-service after a successful message completion.
- `orders_failed_total`, incremented by payments-service when a message fails processing.

11.2 GRAFANA DASHBOARD

The workload overlay includes a Grafana dashboard ConfigMap labeled `grafana_dashboard=1`. The dashboard, `asbdemo` overview, refreshes every 30 seconds and includes panels for order publish rate, order process rate, failure rate, Service Bus active queue depth, throughput comparison, HTTP 5xx rate, and JVM heap utilization.

11.3 PROMETHEUS ALERTS

Alert | Condition | Intent

`AsbdemoHighErrorRate` | 5xx ratio above 1% for five minutes | Detect API instability.

`AsbdemoHighLatency` | p95 HTTP latency above 500 ms for five minutes | Detect slow user-facing requests.

`AsbdemoJvmHeapHigh` | Heap usage above 90% for ten minutes | Detect memory pressure in the JVM.

`AsbdemoNoOrdersPublished` | No order publishes in 15 minutes during business hours | Catch unexpected traffic silence.

`AsbdemoOrdersFailedSpike` | Payment failure rate above 0.5 orders per second for five minutes | Catch consumer-side processing problems.

11.4 TRACING WITH JAEGER

The platform installs Jaeger chart version 4.7.0 in all-in-one mode with memory storage and OTLP enabled. Application containers include the OpenTelemetry Java agent and send traces to Jaeger over OTLP gRPC. This gives request and client spans without adding tracing code directly to the Spring Boot services.

11.5 LOGGING WITH ECK, ELASTICSEARCH, KIBANA, AND FLUENTD

The platform installs the ECK operator and then applies local Elasticsearch/Kibana/Fluentd resources. Elasticsearch runs version 8.16.1 as a single node with a 5 Gi persistent volume and JVM heap set to 512 Mi. Kibana runs version 8.16.1 and connects to that Elasticsearch cluster.

Fluentd runs as a DaemonSet on the AKS user node pool. It tails `/var/log/containers/*.log`, parses CRI log format, enriches records with Kubernetes metadata, attempts to parse the Spring JSON log message, and ships records to Elasticsearch using logstash-style indices with prefix `asbdemo`. It reads the Elasticsearch password from the ECK-generated `elasticsearch-es-elastic-usersecret`.

12. CI/CD AUTOMATION

12.1 GITHUB ACTIONS APPLICATION PIPELINE

The `app-ci` workflow runs on pull requests and pushes to main that touch application files. It computes a seven-character image tag from the Git SHA, builds both Docker images with Docker Buildx, scans them with Trivy for HIGH and CRITICAL vulnerabilities, logs in to Azure using OIDC on pushes, pushes images to ACR, and then updates the GitOps Kustomize image tags.

The GitOps bump job installs Kustomize 5.4.3, runs `kustomize edit set image` for both services, commits `gitops/workloads/envs/dev/kustomization.yaml`, and pushes back to the repository. This commit is the handoff point from CI to ArgoCD.

12.2 GITHUB ACTIONS TERRAFORM PIPELINE

The Terraform workflow runs on infrastructure changes. It performs `terraform fmt -check`, initializes without backend for validation, validates the stack, runs a full plan through HCP Terraform, and posts the plan outcome to a pull request or commit. Applies run only on pushes to main and are protected by the `azure-dev` GitHub environment.

12.3 AZURE DEVOPS PIPELINE MIRRORS

The repository also includes Azure DevOps equivalents. The app pipeline builds, scans, pushes to ACR, fetches a GitHub PAT from Key Vault, and commits the GitOps image bump. The Terraform pipeline installs Terraform, fetches the HCP Terraform token from Key Vault, validates, plans, and applies from an environment-gated deployment job. These files provide an alternate CI/CD implementation while preserving the same delivery model.

13. SECURITY AND IDENTITY MODEL

The strongest security design in the project is the use of short-lived federated identities rather than static secrets for core automation and application access.

- HCP Terraform can authenticate to Azure through OIDC federation, as described in the runbook.
- GitHub Actions uses Azure OIDC login for ACR push operations.
- AKS has OIDC issuer and workload identity enabled.
- Each application pod uses a dedicated Kubernetes service account annotated with the correct

Azure managed identity client ID.

- Service Bus access is split by role: sender for orders, receiver for payments.
- ACR admin user is disabled, and AKS image pull uses the kubelet identity with AcrPull.
- Spring services use DefaultAzureCredential, allowing the same code to work in local service principal mode and AKS workload identity mode.

Dev-oriented tradeoffs are visible in the code: Key Vault public access is enabled, Service Bus public access and local auth are enabled, Grafana uses a static admin password of admin, and several services are ClusterIP rather than externally secured endpoints. Those choices are understandable for a demo/dev stack but should be tightened for shared or production environments.

14. OPERATIONAL RUNBOOK

14.1 FIRST-TIME INFRASTRUCTURE FLOW

- Create Azure AD app registration and service principal for Terraform automation.
- Grant Contributor and User Access Administrator at subscription scope, as required for resource creation and role assignments.
- Create federated credentials for HCP Terraform plan and apply subjects.
- Configure HCP Terraform workspace environment
variables: TFC_AZURE_PROVIDER_AUTH, TFC_AZURE_RUN_CLIENT_ID, ARM_SUBSCRIPTION_ID, and ARM_TENANT_ID
- Set the HCP Terraform working directory to infra/envs/azure-dev.
- Run terraform init, terraform plan, and terraform apply.

14.2 ARGOCD BOOTSTRAP

The bootstrap script installs ArgoCD chart version 7.6.12 into namespace argocd using the provided values file. It then applies two root applications:

- workloads-dev, pointing to gitops/workloads/envs/dev.
- platform-root, pointing to gitops/platform/apps.

Both applications use automated sync, pruning, and self-healing. The workload application also uses CreateNamespace=true.

14.3 USEFUL OPERATOR COMMANDS

```
az aks get-credentials -g rg-asbdemo-dev-sin -n aks-asbdemo-dev-sin --overwrite-existing
```

```
kubectl -n argocd get applications
```

```
kubectl -n asbdemo get rollout,pods,svc
```

```
kubectl -n asbdemo describe rollout orders-service
```

```
kubectl -n asbdemo describe rollout payments-service
```

```
kubectl -n monitoring port-forward svc/kube-prometheus-stack-grafana 3000:80
```

```
kubectl -n jaeger port-forward svc/jaeger-query 16686:16686
```

```
kubectl -n elastic-system port-forward svc/kibana-kb-http 5601:5601
```

15. CURRENT IMPLEMENTATION NOTES AND RECOMMENDED NEXT STEPS

Validation performed while generating this document: the workload, platform apps, and elastic kustomizations all rendered successfully with `kubectl kustomize`.

- No Java unit or integration tests are currently present under `app/**/src/test`. Adding controller tests, serialization tests, and Service Bus client boundary tests would improve confidence.
- The README mentions cert-manager and TLS, and cert-manager is installed, but the current workload manifests only define an HTTP listener and no ClusterIssuer, Certificate, or HTTPS Gateway listener.
- The Gateway and platform services are mostly ClusterIP, which is appropriate for a private dev setup. External access requires additional load balancer or ingress exposure work.
- Service Bus local auth and public network access are enabled in Terraform. For production, consider disabling local auth after confirming Azure AD-only access, and restrict network access through private endpoints or firewall rules.
- Kubernetes ServiceAccount annotations currently contain concrete client IDs, and workload manifests contain concrete Service Bus FQDN and ACR names. A stronger multi-environment design would template these through overlays or generated configuration.
- Grafana uses the demo password `admin` and has no persistence. For longer-lived environments, move credentials to a secret manager and enable persistent storage.
- The Grafana dashboard references `servicebus_queue_active_messages`. That metric requires an exporter or Azure Monitor integration to be present; the current codebase does not define a Service Bus exporter.
- The Terraform Key Vault is provisioned and used by Azure DevOps pipeline documentation, but the Kubernetes workloads are not yet wired to Key Vault CSI in the current manifests.

CONCLUSION

The repository implements a coherent cloud-native platform demonstration: infrastructure is reproducible, application delivery follows GitOps, workloads use managed identity to access Service Bus, rollouts are progressive, and observability spans metrics, logs, dashboards, alerts, and traces. The project is already strong as a dev/demo reference architecture. The biggest next steps for production hardening are test coverage, TLS and external gateway exposure, secret and credential hardening, and making environment-specific values less hardcoded in workload manifests.

Generated from repository code and manifests on 2026-05-02.